

Actividad de Aprendizaje 6: Modelado de Servicios: De SOA a REST, Microservicios y
Serverless

Análisis y desarrollo de Software

Cristian Fabian Tunaroza Rodriguez

José Edwin Gonzales Retamozo

Ficha: 3409609

Servicio Nacional de Aprendizaje - Centro de Biotecnología Agropecuario

25 de septiembre de 2026

Actividad de Aprendizaje 6: Modelado de Servicios: De SOA a REST, Microservicios y Serverless

- **Denominación:** Modelado de interfaces, contratos de comunicación y diseño de servicios de software mediante enfoques SOA, REST, GraphQL, Microservicios y arquitecturas Serverless.
- **Duración:** 6 horas (4 horas de trabajo directo con instructor / 2 horas de trabajo autónomo).
- **Modalidad:** Presencial / Laboratorio de Desarrollo.

A. Descripción de la Actividad

Estimado Aprendiz:

En el ecosistema tecnológico actual, las aplicaciones raras vez funcionan de manera aislada. Los sistemas empresariales modernos son ecosistemas conectados que intercambian datos constantemente a través de la red. Para lograr esto, la ingeniería de software evolucionó desde las pesadas **Arquitecturas Orientadas a Servicios (SOA)** basadas en XML hacia el estándar flexible de las **APIs REST** y la precisión de **GraphQL**. El siguiente gran salto consistió en romper las aplicaciones monolíticas para distribuir las en **Microservicios** independientes y funciones **Serverless** (sin servidor) que se ejecutan bajo demanda en la nube.



En esta actividad, usted aprenderá a diseñar y estructurar la comunicación entre sistemas. Diseñará contratos de comunicación modernos (endpoints de APIs) y analizará mediante un cuadro técnico comparativo las ventajas operativas de descentralizar un sistema de software.

B. Desarrollo Metodológico (Dos Momentos Técnicos)

Momento 1: Cuadro Comparativo de Paradigmas de Servicios

El aprendiz investigará las especificaciones técnicas de cada paradigma para diligenciar una matriz técnica de evaluación arquitectónica, clave para la toma de decisiones:

Criterio Técnico	SOA (Servicios Tradicionales)	API REST	GraphQL	Microservicios / Serverless
Formato de Datos	Principalmente XML	JSON (Ocasionalmente XML o texto plano)	JSON para las respuestas	Generalmente JSON

Protocolo Base	HTTP/HTTPS, SMTP o JMS (usando SOAP/XML-RPC)	HTTP/HTTPS usando GET, POST, PUT, DELETE.	HTTP/HTTPS, habitualmente mediante peticiones POST	HTTP/HTTPS, gRPC o eventos nativos de la nube
-----------------------	--	---	--	---

Criterio Técnico	SOA (Servicios Tradicionales)	API REST	GraphQL	Microservicios / Serverless
Manejo de Consultas	Rígido, basado en operaciones definidas en contratos WSDL fijos	Basado en endpoints fijos; el servidor determina qué datos devuelve cada URL	Flexible y eficiente; el cliente especifica exactamente qué campos y datos necesita	Descentralizado; cada microservicio gestiona sus propias consultas y fuentes de datos
Nivel de Escalamiento	Moderado/Alto; escala a nivel de servicios empresariales, pero puede ser pesado y complejo	Horizontal por instancias, agregando servidores o contenedores	Alto; reduce tráfico redundante, aunque requiere optimizar consultas pesadas	Muy alto/Elástico; permite escalamiento automático por función o servicio

Momento 2: Taller Técnico — Diseño de Contrato de API (REST vs GraphQL)

A partir del informe de análisis de un **Sistema de Entrega de Pedidos a Domicilio**, los aprendices diseñarán los contratos de servicios para la entidad `Pedido` (compuesta por: id, cliente, dirección, estado, lista de productos y repartidor asignado).

1. Diseño en Enfoque REST (Modelado de Endpoints y Verbos HTTP): Los aprendices estructurarán las rutas lógicas del recurso utilizando correctamente los métodos HTTP:

- `POST /api/v1/pedidos` -> Crear un nuevo pedido (Envía JSON en el cuerpo).
- `GET /api/v1/pedidos` -> Listar todos los pedidos.
- `GET /api/v1/pedidos/{id}` -> Consultar el detalle de un pedido específico.
- `PUT /api/v1/pedidos/{id}` -> Actualizar el pedido completo o cambiar su estado.

- o DELETE /api/v1/pedidos/{id} -> Cancelar/eliminar un pedido.

Verbo HTTP	Ruta / Endpoint	Descripción de la Acción
POST	/api/v1/pedidos	Crear un nuevo pedido en el sistema.
GET	/api/v1/pedidos	Obtener la lista completa de pedidos.
GET	/api/v1/pedidos/{id}	Consultar la información detallada de un pedido específico por su ID.
PUT	/api/v1/pedidos/{id}	Actualizar un pedido existente o cambiar su estado.
DELETE	/api/v1/pedidos/{id}	Cancelar o eliminar un pedido por su ID.

Ejemplo

```
1 //CREAR NUEVO PEDIDO
2 // SOLICITUD
3 {
4   "cliente": "Carlos Mendoza",
5   "direccion": "Calle 45 # 12-34, Bogotá",
6   "productos": [
7     {
8       "id": 101,
9       "nombre": "Hamburguesa Doble Carne",
10      "cantidad": 2,
11      "costo": 25000
12    },
13    {
14      "id": 204,
15      "nombre": "Gaseosa Personal 500ml",
16      "cantidad": 2,
17      "costo": 5000
18    }
19  ]
20 }
21
22 // RESPUESTA
23 {
24   "id": "PED-2026-891",
25   "cliente": "Carlos Mendoza",
26   "direccion": "Calle 45 # 12-34, Bogotá",
27   "estado": "EN_PREPARACION",
28   "productos": [
29     {
30       "id": 101,
31       "nombre": "Hamburguesa Doble Carne",
32       "cantidad": 2,
33       "costo": 25000
34     },
35     {
36       "id": 204,
37       "nombre": "Gaseosa Personal 500ml",
38       "cantidad": 2,
39       "costo": 5000
40     }
41   ],
42   "repartidor": null
43 }
44
```

2. Diseño en Enfoque GraphQL (Estructuración del Esquema / Schema): El aprendiz modelará el mismo recurso usando tipos y consultas dirigidas para evitar la sobrecarga de datos:

```
type Pedido {  
  id: ID!  
  cliente: String!  
  total: Float!  
  estado: String!  
}  
type Query {  
  obtenerPedido(id: ID!): Pedido  
}
```

```
1  type Pedido {  
2    id: ID  
3    cliente: String  
4    estado: String  
5    direccion: String  
6    productos: productos  
7    repartidor: repartidor  
8  }  
9  
10 type productos {  
11   id: ID  
12   nombre: String  
13   tipo: String  
14   costo: Int  
15   stock: Int  
16   descuento: Float  
17 }  
18  
19 type repartidor {  
20   nombre: String  
21   id: ID  
22   placa_vehiculo: Int  
23 }  
24  
25 type Query {  
26   obtenerPedido(id: ID): Pedido  
27   obtenerProducto(id: ID): productos  
28   obtenerRepartidor(id: ID): repartidor  
29   enviarPedido(id: ID): repartidor  
30 }  
31  
32 type Mutation {  
33   crearPedido(cliente: String, estado: String, direccion: String, productos: ID, repartidor: ID): Pedido  
34   eliminarPedido(id: ID): Pedido  
35   crearProducto(nombre: String, tipo: String, costo: Int, stock: Int, descuento: Float): productos  
36   eliminarProducto(id: ID): productos  
37 }
```